TP6 : Infrastructure as Code avec Terraform (et intégration Vagrant + Ansible)

Partie_0 : Installation de Docker et Terraform

1. Installer Docker :

```
arij@vagrant:~$ sudo apt install -y docker.io
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
docker.io est déjà la version la plus récente (29.1.3-0ubuntu3~24.04.2).
0 mis à jour, 0 nouvellement installés, 0 à enlever et 76 non mis à jour.
```

2.Installer Terraform :

```
arij@vagrant:~$ sudo apt install -y terraform
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Les NOUVEAUX paquets suivants seront installés :
  terraform
0 mis à jour, 1 nouvellement installés, 0 à enlever et 76 non mis à jour.
Il est nécessaire de prendre 37,7 Mo dans les archives.
Après cette opération, 129 Mo d'espace disque supplémentaires seront utilisés.
Réception de :1 https://apt.releases.hashicorp.com noble/main amd64 terraform amd64 1.17.0-1 [37,7 MB]
37,7 Mo réceptionnés en 19s (1 991 ko/s)
Sélection du paquet terraform précédemment désélectionné.
(Lecture de la base de données... 137658 fichiers et répertoires déjà installés.)
Préparation du dépaquetage de .../terraform_1.17.0-1_amd64.deb ...
Dépaquetage de terraform (1.17.0-1) ...
Paramétrage de terraform (1.17.0-1) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
arij@vagrant:~$ terraform version
Terraform v1.17.0
on linux_amd64

arij@vagrant:~$
```

Partie_1 : Démo Terraform (Docker Local) :

1.Créer le dossier et le fichier main.tf

```
arij@vagrant:~$ mkdir -p ~/terraform/docker-nginx && cd ~/terraform/docker-nginx
arij@vagrant:~/terraform/docker-nginx$ nano main.tf
```

```
  GNU nano 7.2                                                  main.tf *
terraform {
  required_providers {
    docker = {
      source  = "kreuzwerker/docker"
      version = "~> 3.0"
    }
  }
}

provider "docker" {}

resource "docker_image" "nginx" {
  name         = "nginx:latest"
  keep_locally = false
}

resource "docker_container" "nginx" {
  image = docker_image.nginx.image_id
  name  = "nginx_terraform"
  ports {
    internal = 80
    external = 8001
  }
}
```

2. Lancer le cycle Terraform

*Initialiser le dossier de travail +Formater et valider le code

```
arij@vagrant:~/terraform/docker-nginx$ terraform init
Initializing the backend...

Initializing provider plugins...
- Finding kreuzwerker/docker versions matching "~> 3.0"...
- Installing kreuzwerker/docker v3.9.0...
- Installed kreuzwerker/docker v3.9.0 (self-signed, key ID 0DCE698927DAF8EC)
Partner and community providers are signed by their developers.
If you'd like to know more about provider signing, you can read about it here:
https://developer.hashicorp.com/terraform/cli/plugins/signing

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
arij@vagrant:~/terraform/docker-nginx$ terraform fmt
arij@vagrant:~/terraform/docker-nginx$ terraform validate
Success! The configuration is valid.
```

*Afficher les actions que Terraform va effectuer

```
arij@vagrant:~/terraform/docker-nginx$ terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
  + create

Terraform will perform the following actions:

  # docker_container.nginx will be created
  + resource "docker_container" "nginx" {
      + attach                                      = false
      + bridge                                      = (known after apply)
      + command                                     = (known after apply)
      + container_logs                              = (known after apply)
      + container_read_refresh_timeout_milliseconds = 15000
      + entrypoint                                  = (known after apply)
      + env                                         = (known after apply)
      + exit_code                                   = (known after apply)
      + hostname                                    = (known after apply)
      + id                                          = (known after apply)
      + image                                       = (known after apply)
      + init                                        = (known after apply)
      + ipc_mode                                    = (known after apply)
      + log_driver                                  = (known after apply)
      + logs                                        = false
      + memory_reservation                          = 0
      + must_run                                    = true
      + name                                        = "nginx_terraform"
      + network_data                                = (known after apply)
      + network_mode                                = "bridge"
      + read_only                                   = false
      + remove_volumes                              = true
      + restart                                     = "no"
```

*Créer les ressources

```
arij@vagrant:~/terraform/docker-nginx$ terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
  + create

Terraform will perform the following actions:

  # docker_container.nginx will be created
  + resource "docker_container" "nginx" {
      + attach                                      = false
      + bridge                                      = (known after apply)
      + command                                     = (known after apply)
      + container_logs                              = (known after apply)
      + container_read_refresh_timeout_milliseconds = 15000
      + entrypoint                                  = (known after apply)
      + env                                         = (known after apply)
      + exit_code                                   = (known after apply)
      + hostname                                    = (known after apply)
```

```
docker_image.nginx: Creating...
docker_image.nginx: Still creating... [00m10s elapsed]
docker_image.nginx: Still creating... [00m20s elapsed]
docker_image.nginx: Creation complete after 28s [id=sha256:4c41e2d0a3a55b325ed572cd0200bdb72af1433c3daa2bca6ef8eea68ce504a2nginx:late
st]
docker_container.nginx: Creating...
docker_container.nginx: Creation complete after 2s [id=1f0368361faecb5265dcc81f9977c169e7eb4d750dbbe561e42e47ab7cd051cf]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
arij@vagrant:~/terraform/docker-nginx$
```

3. Vérifier le résultat

```
arij@vagrant:~/terraform/docker-nginx$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED         STATUS         PORTS                    NAMES
1f0368361fae   4c41e2d0a3a5   "/docker-entrypoint.…"   5 minutes ago   Up 5 minutes   0.0.0.0:8001->80/tcp   nginx_terraform
```

```
arij@vagrant:~/terraform/docker-nginx$ curl http://localhost:8001
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

```
arij@vagrant:~/terraform/docker-nginx$ terraform state list
docker_container.nginx
docker_image.nginx
arij@vagrant:~/terraform/docker-nginx$ terraform show
# docker_container.nginx:
resource "docker_container" "nginx" {
    attach                                      = false
    bridge                                      = null
    command                                     = [
        "nginx",
        "-g",
        "daemon off;",
    ]
    container_read_refresh_timeout_milliseconds = 15000
    cpu_set                                     = null
    cpu_shares                                  = 0
    domainname                                  = null
    entrypoint                                  = [
        "/docker-entrypoint.sh",
    ]
    env                                         = []
    hostname                                    = "1f0368361fae"
    id                                          = "1f0368361faecb5265dcc81f9977c169e7eb4d750dbbe561e42e47ab7cd051cf"
    image                                       = "sha256:4c41e2d0a3a55b325ed572cd0200bdb72af1433c3daa2bca6ef8eea68ce504a2"
    init                                        = false
    ipc_mode                                    = "private"
    log_driver                                  = "json-file"
    logs                                        = false
    max_retry_count                             = 0
    memory                                      = 0
    memory_reservation                          = 0
    memory_swap                                 = 0
    must_run                                    = true
```

*Les Variables :

1.Créer le fichier variables

```
  GNU nano 7.2                                                variables.tf *
variable "external_port" {
  description = "Port de l'hôte associé au port 80 du conteneur nginx"
  type        = number
  default     = 8001
}
```

2.Modifier main.tf :

```
  GNU nano 7.2                                                  main.tf *
terraform {
  required_providers {
    docker = {
      source  = "kreuzwerker/docker"
      version = "~> 3.0"
    }
  }
}

provider "docker" {}

resource "docker_image" "nginx" {
  name         = "nginx:latest"
  keep_locally = false
}

resource "docker_container" "nginx" {
  image = docker_image.nginx.image_id
  name  = "nginx_terraform"
  ports {
    internal = 80
    external = var.external_port
  }
}
```

3.relancer :

```
docker_image.nginx: Creating...
docker_image.nginx: Still creating... [00m10s elapsed]
docker_image.nginx: Creation complete after 20s [id=sha256:4c41e2d0a3a55b325ed572cd0200bdb72af1433c3daa2bca6ef8eea68ce504a2nginx:late
st]
docker_container.nginx: Creating...
docker_container.nginx: Creation complete after 1s [id=073659dda6780517508b2f70b04d0c5cf502d9a4a0c839bee34d00cdebbe5f10]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
arij@vagrant:~/terraform/docker-nginx$
```

4. Recréer le conteneur en changeant le port via la ligne de commande

```
arij@vagrant:~/terraform/docker-nginx$ terraform apply -var external_port=8002

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
  + create

Terraform will perform the following actions:

  # docker_container.nginx will be created
  + resource "docker_container" "nginx" {
      + attach                                      = false
      + bridge                                      = (known after apply)
      + command                                     = (known after apply)
      + container_logs                              = (known after apply)
      + container_read_refresh_timeout_milliseconds = 15000
      + entrypoint                                  = (known after apply)
      + env                                         = (known after apply)
      + exit_code                                   = (known after apply)
      + hostname                                    = (known after apply)
```

```
docker_image.nginx: Creating...
docker_image.nginx: Still creating... [00m10s elapsed]
docker_image.nginx: Creation complete after 18s [id=sha256:4c41e2d0a3a55b325ed572cd0200bdb72af1433c3daa2bca6ef8eea68ce504a2nginx:late
st]
docker_container.nginx: Creating...
docker_container.nginx: Creation complete after 1s [id=ddef2ec3514f793584e926a9bbfef961a74f60702b0929f8315efb6449ebff5b]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
arij@vagrant:~/terraform/docker-nginx$
```

*Docker Distant (via SSH) :

1.Autoriser la clé SSH :

```
arij@vagrant:~/terraform/docker-nginx$ ssh-copy-id arij@192.168.159.22
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/arij/.ssh/id_ed25519.pub"
The authenticity of host '192.168.159.22 (192.168.159.22)' can't be established.
ED25519 key fingerprint is SHA256:Gju9LVU+yTcTncyBlHGtLtbaiBdIvPgjnyRdFyVYbdg.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys
arij@192.168.159.22's password:

Number of key(s) added: 1

Now try logging into the machine, with:   "ssh 'arij@192.168.159.22'"
and check to make sure that only the key(s) you wanted were added.
```

2. Vérifier la connexion et Docker

```
arij@vagrant:~/terraform/docker-nginx$ ssh arij@192.168.159.22 "docker ps"
CONTAINER ID   IMAGE                                                                COMMAND                  CREATED          STATUS
                  PORTS
                        NAMES
816e53614e13   coollabsio/sentinel:1.1.2                                            "/app/sentinel"          12 seconds ago   Up 12 se
conds (healthy)
                        coolify-sentinel
5f0f7ff55db2   webapp-web                                                           "/docker-entrypoint.…"   47 hours ago     Up About
 a minute             0.0.0.0:8082->80/tcp, [::]:8082->80/tcp
                        webapp-web-1
d0b76c271a4f   jenkins/jenkins:lts                                                  "/usr/bin/tini -- /u…"   47 hours ago     Up About
 a minute             50000/tcp, 0.0.0.0:8081->8080/tcp
                        jenkins
508b538e7a1f   portainer/portainer-ce:latest                                        "/portainer"             2 days ago       Up About
 a minute             8000/tcp, 9443/tcp, 0.0.0.0:9000->9000/tcp
                        portainer
4a11ccdec9c1   8utajpy38ykg5zylpyjwhqn6:77c0125db2479c75067a3223cda6558485ba05cb   "/bin/bash -l -c 'np…"   5 days ago       Up About
 a minute             3000/tcp
                        8utajpy38ykg5zylpyjwhqn6-20261017T151535
99fbb772fbe2   codeberg.org/forgejo/forgejo:16                                      "/usr/bin/entrypoint…"   5 days ago       Up About
 a minute             0.0.0.0:2222->22/tcp, [::]:2222->22/tcp, 0.0.0.0:3001->3000/tcp, [::]:3001->3000/tcp
                        forgejo
141b6a15315a   traefik:v3.7                                                         "/entrypoint.sh --pi…"   5 days ago       Up About
 a minute (healthy)
                        coolify-proxy
ccf7dfd994d1   coollabsio/coolify:4.4.6                                             "docker-php-serversi…"   5 days ago       Up About
 a minute (healthy)   8000/tcp, 8443/tcp, 0.0.0.0:6001-6002->6001-6002/tcp, [::]:6001-6002->6001-6002/tcp, 9000/tcp, 0.0.0.0:8000->80
80/tcp, [::]:8000->8080/tcp   coolify
068972ac4dd6   postgres:15-alpine                                                   "docker-entrypoint.s…"   5 days ago       Up About
 a minute (healthy)   5432/tcp
                        coolify-db
180cf1707501   redis:7-alpine                                                       "docker-entrypoint.s…"   5 days ago       Up About
```

3.Lancer Terraform :

```
arij@vagrant:~/terraform/docker-nginx$ terraform init
Initializing the backend...

Initializing provider plugins...
- Reusing version 3.9.0 of kreuzwerker/docker from the dependency lock file
- Using previously-installed kreuzwerker/docker v3.9.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
arij@vagrant:~/terraform/docker-nginx$ terraform apply
docker_image.nginx: Refreshing state... [id=sha256:4c41e2d0a3a55b325ed572cd0200bdb72af1433c3daa2bca6ef8eea68ce504a2nginx:latest]
docker_container.nginx: Refreshing state... [id=ddef2ec3514f793584e926a9bbfef961a74f60702b0929f8315efb6449ebff5b]
```

4.Vérification :

```
arij@vagrant:~/terraform/docker-nginx$ ssh arij@192.168.159.22 "docker ps"
CONTAINER ID   IMAGE                                                                COMMAND                  CREATED         STATUS
               PORTS
                         NAMES
14ca35ca8197   4c41e2d0a3a5                                                         "/docker-entrypoint.…"   4 minutes ago   Up 4 minu
tes             0.0.0.0:8080->80/tcp
                         nginx_terraform_distant
816e53614e13   coollabsio/sentinel:1.1.2                                            "/app/sentinel"          7 minutes ago   Up 7 minu
tes (healthy)
                         coolify-sentinel
5f0f7ff55db2   webapp-web                                                           "/docker-entrypoint.…"   47 hours ago    Up 8 minu
tes             0.0.0.0:8082->80/tcp, [::]:8082->80/tcp
                         webapp-web-1
d0b76c271a4f   jenkins/jenkins:lts                                                  "/usr/bin/tini -- /u…"   2 days ago      Up 8 minu
tes             50000/tcp, 0.0.0.0:8081->8080/tcp
                         jenkins
508b538e7a1f   portainer/portainer-ce:latest                                        "/portainer"             2 days ago      Up 8 minu
tes             8000/tcp, 9443/tcp, 0.0.0.0:9000->9000/tcp
                         portainer
4a11ccdec9c1   8utajpy38ykg5zylpyjwhqn6:77c0125db2479c75067a3223cda6558485ba05cb     "/bin/bash -l -c 'np…"   5 days ago      Up 8 minu
tes             3000/tcp
                         8utajpy38ykg5zylpyjwhqn6-20261017T151535
99fbb772fbe2   codeberg.org/forgejo/forgejo:16                                      "/usr/bin/entrypoint…"   5 days ago      Up 8 minu
tes             0.0.0.0:2222->22/tcp, [::]:2222->22/tcp, 0.0.0.0:3001->3000/tcp, [::]:3001->3000/tcp
                         forgejo
141b6a15315a   traefik:v3.7                                                         "/entrypoint.sh --pi…"   5 days ago      Up 8 minu
tes (healthy)
                         coolify-proxy
ccf7dfd994d1   coollabsio/coolify:4.4.6                                             "docker-php-serversi…"   5 days ago      Up 8 minu
tes (healthy)   8000/tcp, 8443/tcp, 0.0.0.0:6001-6002->6001-6002/tcp, [::]:6001-6002->6001-6002/tcp, 9000/tcp, 0.0.0.0:8000->8080/tcp
, [::]:8000->8080/tcp   coolify
068972ac4dd6   postgres:15-alpine                                                   "docker-entrypoint.s…"   5 days ago      Up 8 minu
tes (healthy)   5432/tcp
```

Partie_2 : Le Lab d'intégration complet (Vagrant + Ansible + Terraform) :

1. Créer l'image Docker du CV

```
arij@vagrant:~/terraform/docker-nginx$ mkdir -p ~/cv-image && cd ~/cv-image
arij@vagrant:~/cv-image$ nano index.html
arij@vagrant:~/cv-image$ nano Dockerfile
arij@vagrant:~/cv-image$ sudo docker build -t mon-cv:latest .
[sudo] password for arij:
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  3.072kB
Step 1/2 : FROM nginx:alpine
alpine: Pulling from library/nginx
b9b388a21ecf: Pulling fs layer
657023555a2a: Pulling fs layer
26185d08a3ec: Pulling fs layer
7cc330898115: Pulling fs layer
7b229517ed22: Pulling fs layer
12b85cff06fd: Pulling fs layer
2ec5fb117ffa: Pulling fs layer
9b0f7dfe35b2: Pulling fs layer
657023555a2a: Download complete
26185d08a3ec: Download complete
7cc330898115: Download complete
2ec5fb117ffa: Download complete
9b0f7dfe35b2: Download complete
7b229517ed22: Download complete
a4d5b1f57abc: Download complete
7b229517ed22: Pull complete
acc2d54c3743: Download complete
12b85cff06fd: Download complete
12b85cff06fd: Pull complete
657023555a2a: Pull complete
26185d08a3ec: Pull complete
```

2. Créer les 2 VMs avec Vagrant :

```
arij@vagrant:~/tp6-lab$ vagrant status
Current machine states:

vm1                       running (virtualbox)
vm2                       running (virtualbox)

This environment represents multiple VMs. The VMs are all listed
above with their current state. For more information about a specific
VM, run `vagrant status NAME`.
arij@vagrant:~/tp6-lab$
```

3.Ansible : Installer Docker sur les VMs

```
arij@vagrant:~/tp6-lab$ ansible-playbook -i hosts.ini playbook.yml

PLAY [Installer Docker] ********************************************************************************************************

TASK [Gathering Facts] *********************************************************************************************************
ok: [vm1]
ok: [vm2]

TASK [Installer Docker] ********************************************************************************************************
ok: [vm1]
changed: [vm2]

TASK [Démarrer Docker] *********************************************************************************************************
ok: [vm2]
ok: [vm1]

TASK [Ajouter l'utilisateur vagrant au groupe docker] **************************************************************************
ok: [vm1]
changed: [vm2]

PLAY RECAP *********************************************************************************************************************
vm1                        : ok=4    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
vm2                        : ok=4    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

arij@vagrant:~/tp6-lab$
```

4.Terraform : Déployer le CV

```
arij@vagrant:~/tp6-lab$ mkdir terraform && cd terraform
arij@vagrant:~/tp6-lab/terraform$ nano main.tf
arij@vagrant:~/tp6-lab/terraform$
```

```
arij@vagrant:~/tp6-lab/terraform$ terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
  + create

Terraform will perform the following actions:

  # docker_container.cv_vm1 will be created
  + resource "docker_container" "cv_vm1" {
      + attach                                      = false
      + bridge                                      = (known after apply)
      + command                                     = (known after apply)
      + container_logs                              = (known after apply)
      + container_read_refresh_timeout_milliseconds = 15000
      + entrypoint                                  = (known after apply)
      + env                                         = (known after apply)
      + exit_code                                   = (known after apply)
      + hostname                                    = (known after apply)
      + id                                          = (known after apply)
      + image                                       = (known after apply)
      + init                                        = (known after apply)
```

```
docker_image.cv_vm2: Creating...
docker_image.cv_vm1: Creating...
docker_image.cv_vm2: Still creating... [00m10s elapsed]
docker_image.cv_vm1: Still creating... [00m10s elapsed]
docker_image.cv_vm2: Still creating... [00m20s elapsed]
docker_image.cv_vm1: Still creating... [00m20s elapsed]
docker_image.cv_vm1: Creation complete after 30s [id=sha256:8f3025a7f4fed06278c2738101fc30b6dda9e97d6ad9380973c8b8efe9e0eaaemon-cv:la
test]
docker_container.cv_vm1: Creating...
docker_image.cv_vm2: Still creating... [00m30s elapsed]
docker_image.cv_vm2: Creation complete after 32s [id=sha256:a7fc0f885b6894deb45be1ac713b64e8d85449420f78eba5e102bee40c598e26mon-cv:la
test]
docker_container.cv_vm2: Creating...
docker_container.cv_vm1: Creation complete after 2s [id=d503a6daf4cf9b904a257f70446a839880ac199d7a145d3ea9c85f953a85a840]
docker_container.cv_vm2: Creation complete after 2s [id=2302aa593d49f300b3f4b51634947893def6a92c2b0f731033bbfe15d9257032]

Apply complete! Resources: 4 added, 0 changed, 0 destroyed.
arij@vagrant:~/tp6-lab/terraform$
```

5. Vérifier dans navigateur :

Vm1 :



CV de Arij - Master DevOps

Vm2 :



CV de Arij - Master DevOps